

Business Rules Syntax Documentation

Business Analyst Guide to JSON Claims Integrity Rules

This document provides a comprehensive guide for business analysts to write claims integrity and risk review rules using our Domain-Specific Language (DSL).

Current Rule Capabilities

- Use one or more `RULE ... END` blocks as a rule set in a single editor run.
- Add plain-English comments with `--` at the start of a line.
- Use quantifiers like `EXISTS`, `FORALL`, and `COUNT(...)`.

Advanced Features (Optional)

These advanced options are intended for occasional use in specialized scenarios.

- **Variable binding:** define reusable aliases for long or repeated field paths.
- **Helper function construction:** encapsulate repeated complex checks into named expressions.

Rule Block Styles

The engine currently accepts both of these rule block styles:

Style A (recommended)

```
RULE <rule_name>
DESCRIPTION "<description>"
WHEN <predicate>
THEN <action>
END
```

Style B (legacy shorthand)

```
RULE <rule_name> "<description>"
WHEN <predicate>
THEN <action>;
```

Rule Structure

Every rule follows this basic structure:

```
RULE <rule_name>
DESCRIPTION "<description>"
WHEN <predicate>
THEN <action>
END
```

- **RULE:** Keyword that starts a rule definition
- **rule_name:** Unique identifier for the rule (letters, numbers, underscores)
- **DESCRIPTION:** Human-readable description in quotes
- **WHEN:** Introduces the condition to check
- **predicate:** Logical expression that evaluates to true or false
- **THEN:** Introduces the action to take when condition is true
- **action:** What to do when the rule matches
- **END:** Ends the rule block

Comments

Use comments for plain-English notes:

```
-- Check for unusually high claim amounts
```

Predicates (Conditions)

Comparison Operators

```
field = value           # Equals
field != value          # Not equals
field > value           # Greater than
field < value           # Less than
field >= value          # Greater than or equal
field <= value          # Less than or equal
```

Null Checks

```
field IS NULL           # Check if field has no value
field IS NOT NULL       # Check if field has a value
```

Logical Operators

```
predicate1 AND predicate2 # Both must be true
predicate1 OR predicate2  # At least one must be true
NOT predicate              # Negates the predicate
```

Quantifiers

EXISTS loop.segment WHERE predicate exists	# At least one matching item exists
FORALL loop.segment WHERE predicate predicate	# All items match the predicate
COUNT(loop.segment) > n	# Number of items comparison

String Operations

field CONTAINS "text"	# Field contains the text
field MATCHES "pattern"	# Field matches regex pattern (future)

Field References

Claim JSON Structure

Claims are represented in a hierarchical JSON-like structure: - **Interchange** → **Functional Group** → **Transaction** → **Loops** → **Segments** → **Elements**

Common loops in 837P: - 2300: Claim Information - 2400: Service Line - 2010: Provider/Patient Name - 2320: Other Subscriber Information

Common segments: - CLM: Claim segment - SV1: Professional Service - DTP: Date/Time Period - HI: Health Care Diagnosis Code - NM1: Name

Reference Formats

field_name	# Simple field reference
CLM.claim_amount	# Segment.field
2300.CLM.claim_amount	# Loop.segment.field

Values

String Values

"text in quotes"

Numeric Values

100
250.50
10000.00

Date Values (as strings)

"2026-01-24"

"20260124"

Actions

FLAG_FRAUD

Flag a claim as potentially fraudulent with a reason.

FLAG_FRAUD "Reason for flagging"

REJECT

Reject the claim with a reason.

REJECT "Reason for rejection"

REQUIRE_REVIEW

Require manual review with a note.

REQUIRE_REVIEW "Note for reviewer"

RISK_SCORE

Assign a risk score from 0 to 100.

RISK_SCORE 75

Multiple Actions

Combine multiple actions with brackets and commas.

[FLAG_FRAUD "High amount", RISK_SCORE 85, REQUIRE_REVIEW "Urgent"]

Complete Examples

Example 1: High Claim Amount

```
RULE high_claim_amount "Flag unusually high claim amounts"  
WHEN 2300.CLM.claim_amount > 50000.0  
THEN FLAG_FRAUD "Claim amount exceeds $50,000 threshold";
```

Example 2: Missing Required Field

```
RULE missing_diagnosis "Reject claims without diagnosis code"
WHEN 2300.HI.diagnosis_code IS NULL
THEN REJECT "Diagnosis code is required for all claims";
```

Example 3: Complex Pattern with AND

```
RULE excessive_daily_charges "Multiple high-value services same day"
WHEN COUNT(2400) > 10
    AND FORALL 2400 WHERE 2400.SV1.line_charge > 500.0
THEN [FLAG_FRAUD "Excessive billing pattern", RISK_SCORE 80];
```

Example 4: Unbundling Detection

```
RULE unbundling_check "Detect unbundled E&M codes"
WHEN EXISTS 2400.SV1 WHERE 2400.SV1.procedure_code = "99213"
    AND EXISTS 2400.SV1 WHERE 2400.SV1.procedure_code = "99214"
THEN FLAG_FRAUD "Possible unbundling of E&M services";
```

Example 5: Age-Inappropriate Service

```
RULE age_check "Service not appropriate for patient age"
WHEN 2010.NM1.patient_age < 18
    AND 2400.SV1.procedure_code = "99385"
THEN REQUIRE_REVIEW "Adult preventive service for minor";
```

Example 6: Multiple Conditions with OR

```
RULE suspicious_location "Service in unusual location"
WHEN 2400.SV1.place_of_service = "99"
    OR 2400.SV1.place_of_service IS NULL
THEN [REQUIRE_REVIEW "Verify service location", RISK_SCORE 50];
```

Best Practices

1. Use Descriptive Names

✓ GOOD: high_claim_amount
x BAD: rule1

2. Write Clear Descriptions

✓ GOOD: "Flag claims with amounts exceeding \$50,000"
x BAD: "Check amount"

3. Be Specific with Thresholds

- ✓ GOOD: `claim_amount > 50000.0`
- ✗ BAD: `claim_amount > 1000000.0` # Too high, won't catch most fraud

4. Combine Related Checks

- ✓ GOOD:
RULE comprehensive_check "Multiple red flags"
WHEN amount > 50000.0 AND COUNT(2400) > 20
THEN [FLAG_FRAUD "Multiple issues", RISK_SCORE 90];
- ✗ BAD: Creating 10 separate rules for related checks

5. Use Appropriate Risk Scores

- 0-30: Low risk, informational
- 31-60: Medium risk, review recommended
- 61-85: High risk, review required
- 86-100: Critical risk, immediate action

Common Patterns

Pattern: Range Check

```
RULE amount_range "Amount outside normal range"  
WHEN 2300.CLM.claim_amount < 10.0  
      OR 2300.CLM.claim_amount > 100000.0  
THEN REQUIRE_REVIEW "Unusual claim amount";
```

Pattern: Duplicate Detection

```
RULE duplicate_claims "Multiple claims same day"  
WHEN COUNT(2300) > 3  
THEN FLAG_FRAUD "Possible duplicate submission";
```

Pattern: Cross-Field Validation

```
RULE date_consistency "Service date after submission"  
WHEN 2400.DTP.service_date > 2300.DTP.submission_date  
THEN REJECT "Service date cannot be after submission";
```

Pattern: Provider Validation

```
RULE provider_status "Check provider eligibility"  
WHEN 2010.NM1.provider_status = "suspended"
```

```
OR 2010.NM1.provider_status = "terminated"  
THEN REJECT "Provider is not eligible to submit claims";
```

Testing Your Rules

1. Start with simple rules and test them
2. Gradually add complexity
3. Use the web interface to validate syntax
4. Test against sample claims data
5. Review false positives and adjust thresholds

Troubleshooting

Parse Errors

- Check for missing semicolons
- Ensure quotes are balanced
- Verify keyword spelling (RULE, WHEN, THEN)
- Check parentheses are matched

Runtime Errors

- Verify field names match your claim JSON structure
- Ensure numeric comparisons use numbers, not strings
- Check loop IDs are correct (2300, 2400, etc.)

Need Help?

Contact the development team for:

- Custom field mappings
- New operators or functions
- Performance issues
- Questions about claim JSON structure